SAPIENZA
UNIVERSITÀ DI ROMA

Faculty of Information Engineering, Informatics and Statistics
Department of Computer Science

Concurrent Systems

Author:
Simone Lidonnici

7 march 2026

Contents

1 Mutex **1**
1.1 Concurrency . . . 1
1.2 Mutex definition . . . 1
1.3 Atomic Read/Write registers . . . 2
1.3.1 Peterson algorithm . . . 2
1.3.2 Tournament based algorithm . . . 5
1.3.3 Fast mutex algorithm . . . 5
1.3.4 Deadlock freedom to Bounded bypass . . . 7
1.4 Special HW primitives . . . 8
1.4.1 Test&Set . . . 8
1.4.2 Swap . . . 9
1.4.3 Compare&Swap . . . 9
1.4.4 Fetch&Add . . . 10
1.5 Safe registers . . . 10
1.5.1 Bakery algorithm . . . 11
1.5.2 Aravind algorithm . . . 12

1
Mutex

1.1 Concurrency

A Concurrent System is a set of sequential state machines that run simultaneously and interact through a shared medium. The goal is to combine the work of different processes, that solve different tasks in parallel.
We will focus on concurrent systems that have 3 main properties:

- **Reliable**: every process correctly executes its program.
- **Asynchronous**: each process has its own clock.
- **Shared memory**: every process has a local memory but there are some registries that can be accesseb by every process.

The number of processors that we have is not relevant so we will assume we have one for every process.
Synchronization of a concurrent systems means that the behaviour of one process depends on the behaviour of the other processes. There are two principal interactions:

- Cooperation: different processes work to make all succeed in their task.
- Competition: different processes aims to execute the same action, but only some of the succeed. This is usually when trying to access a shared resource.

1.2 Mutex definition

A mutex ensures that a part of the code is executed as if it was atomic, so without other processes running in between. The mutex is needed only in code parts that interact with shared data.

Critical section

A **critical section (CS)** is a set of code parts that must be excuted without interferences. When a process is in a CS, no other process can be in a CS for the same shared object.

The mutex problem is designig an entry protocol **lock** and an exit protocol **unlock**, such that the code encapsulated between this two protocols can be run by at most one process at a time.
Every mutex should satisfy 2 properties:

- **Safety**: there is at most one process in a CS.
- **Liveness**: something good everntually happen. This property has various options:
  - **Deadlock starvation**: if there is at least one invocation of lock, eventually after at least one process enters a CS.
  - **Starvation freedom**: every invocation of lock eventually grants access to the associated CS.
  - **Bounded bypass**: with n processes, there exists $f : N \to N$ such that every lock enters the CS after at most $f(n)$ other CS (processes).

The liveness properties have a hierarchy such that Bounded bypass implies Starvation freedom that also implies Deadlock freedom.

1.3 Atomic Read/Write registers

Atomic Read/Write registers are storage units that can be accessed through two operations (`READ` and `WRITE`) such that:

1. Each invocation of an operation:
   - looks instantaneous and can be considered as a single point on the timeline (there exists a function $t : OpInv \to \mathbb{R}^+$)
   - may be located in any point between its starting and ending time (we have that $t(opInv) \in [t_{start}(opInv), t_{end}(opInv)]$)
   - does not happen together with any other operation ($t(opInv) \neq t(opInv')$)
2. Every `READ` returns the preceeding value written in the register.

There are different types of registers based on wheter they can be read and written by one or many different processes:

- single-read/single-write (SRSW)
- single-read/multiple-write (SRMW)
- multiple-read/single-write (MRSW)
- multiple-read/multiple-write (MRMW)

1.3.1 Peterson algorithm

The Peterson algorithm designs a mutex for 2 processes:

Algorithm: Peterson algorithm (2 processes)

```
// Inizialize FLAG[i] as false
def lock(i):
    FLAG[i] = true
    LAST = i
    wait (FLAG[i-1] = false or LAST != i)
    return

def unlock(i):
    FLAG[i] = false
    return
```

This algorithm requires 2 one-bit MRSW registers for the `FLAG` and 1 one-bit MRMW register for `LAST`. Every lock-unlock requires 5 accesses to the registers.

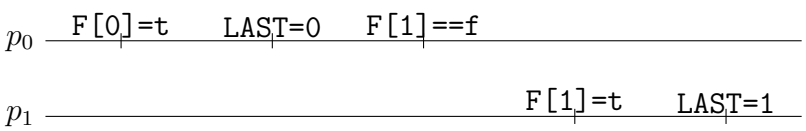
Peterson algorithm properties (2 processes)

The Peterson algorithm designs a working mutex and has the bounded bypass property with bound 1.

Proof of safety:

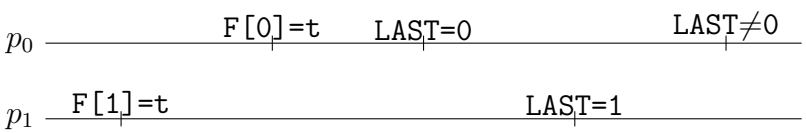
We can prove that the algorithm works by contraddiction, assuming that the two processes p_0 and p_1 are inside the CS. How has p_0 entered the CS:

1. If `FLAG[i]=false` that is possible with this sequence:



   With this sequence p_1 cannot be in the CS because `LAST=1` and `FLAG[0]=true`.

2. If `LAST=1` that is possible with this sequence:



   With this sequence p_1 cannot be in the CS because `LAST=1` and `FLAG[0]=true`.

Proof of liveness:

Let p_0 invock lock, if it wins waits 0, otherwise must be `FLAG[1]=true` and `LAST=0`. `FLAG[1]=true` means that p_1 has invocked the lock and will eventually enter the CS and unlock, putting the flag down. Now p_1 could invock the lock again:

- If it never locks again p_0 will eventually read the flag down and enter the CS, waiting 1.
- If p_1 locks again:
  - If p_0 reads `FLAG[1]` before p_1 locks again, then p_0 enters the CS, waiting 1.
  - If p_0 doesn't read before p_1 locks again, then p_1 will set `LAST=1` and will wait, so p_0 will eventually read `LAST` and enter the CS, waiting 1.

There is also a Peterson algorithm for an arbitrary number of processes n, using levels:

Algorithm: Peterson algorithm (n processes)

```
// Inizialize FLAG[i] as false
def lock(i):
    for lev in [1,n-1] :
        FLAG[i] = lev
        LAST[lev] = i
        wait (∀k FLAG[k] < lev or LAST[lev] != i)
    return

def unlock(i):
    FLAG[i] = 0
    return
```

This algorithm require n MRSW registers with $\lceil \log n \rceil$ bits for `FLAG` and $n-1$ MRSW registers with $\lceil \log n \rceil$ bits for `LAST`. Every lock-unlock requires $(n-1)(n+2)$ accesses to the registers, so has a cost $O(n^2)$.

Peterson algorithm properties (n processes)

For every level $l \in [0, n-1]$, there are at most $n-l$ processes in the level l. This implies the mutex works for $l = n-1$.
The algorithm has a starvation freedom property such that every process that invokes lock and is at level $l \leq n-1$ eventually wins and eneters the CS.

Proof of safety:
Proof by induction on the level l:

1. Base case: $l = 0$
2. Induction: a process p can increase its level by writing its `FLAG` at $l+1$ and its index in `LAST[l+1]`. Let p_x be the last one that writes `LAST[l+1]=x`, for p_x to pass at level $l+1$, it must be that $\forall k$ `FLAG[k]<l+1`. So, p_x is the only proc at level $l+1$.

   Otherwise, p_x is blocked in the wait and so we have at most $n-l-1$ processes at level $l+1$, those at level l, that by induction are at most $n-l$, except for p_x that is blocked in its $(l+1)$-th wait.

Proof of liveness:
Proof by reverse induction on the level l:

1. Base case: $l = n-1$

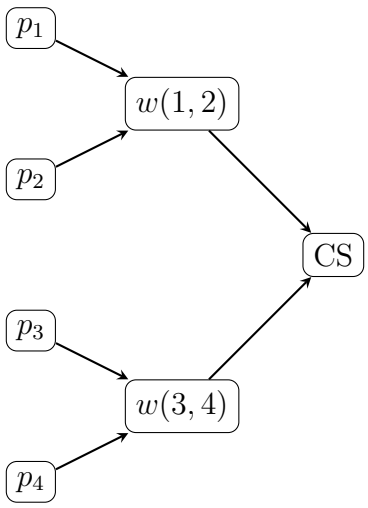
2. Induction: assume a p_x is blocked at level l, so $\exists k$ `FLAG[k]`$\geq$`l+1` $\wedge$ `LAST[l+1]=x`. If some p_y sets `LAST[l+1]=y` p_x will exit from its wait and pass to level $l+1$.

   Otherwise, let $G = \{p_i|$ `FLAG[i]` $\geq$ `l+1`$\}$ and $L = \{p_i|$`FLAG[i]<l+1`$\}$. Every $p \in L$ will never enter its $(l+1)$-th loop and every $p \in G$ will eventually win and move into L. Eventually, p_x will be the only one in its $(l+1)$-th loop, with all other processes at level $l' < l+1$ and p_x will eventually pass to level $l+1$ and win.

The algorithm doesn't satisfy bounded bypass because if a process is sleeping the other can enter the CS an unbounded number of times.

1.3.2 Tournament based algorithm

To reduce the cost of a mutex we can use a tournament betwen pairs of processes:



For the competition between two processes the Peterson algorithm is used and this means that a process enters the CS after $\lceil \log 2 \rceil$ competitions, each of constant cost for a total cost of $O(\log n)$. This algorithm has also a bounded bypass owith bound $\lceil \log n \rceil$.

1.3.3 Fast mutex algorithm

Is it possible to reduce the cost to $O(1)$, using the **Fast mutex** algorithm by Lamport:

Algorithm: Fast mutex

```
// Inizialize Y as ⊥, X as 0
def lock(i):
    FLAG[i]=true// *
    X=i
    if Y!=⊥ :
        FLAG[i]=false
        wait Y = ⊥
        goto *
    else :
        Y=i
        if X=i :
            return
        else :
            FLAG[i]=false
            wait ∀j FLAG[j]=false
            if Y=i :
                return
            else :
                wait Y=⊥
                goto *

def unlock(i):
    Y=⊥
    FLAG[i] = 0
    return
```

This algorithm require 5 accesses to the registers if there is no contention.

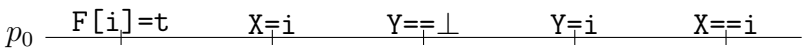
Fast mutex properties

The algorithm designs a correct mutex with the deadlock freedom property.

Proof of safety:

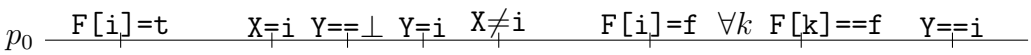
Proof by contraddiction, assuming two processes p_i and p_j are inside the CS. How can p_i enter the CS:

1. In the first if `X=i`:



   For p_j to enter the CS, it must read `Y=⊥`, so it must read `Y` before p_i writes it equal to i. Also p_j must write `X=j` and it must have done it before p_i writes it equal to i, otherwise p_i couldn't have read the value of `X` equal to i. So, when p_j reads `X`, it finds it different from j and cannot enter the CS.

2. In the second if `Y=i`:



For p_j to enter the CS, it must read `Y=`$\perp$, so it must read `Y` before p_i writes it equal to i. Also p_j must write `X=j`, if it has done it before p_i writes it equal to i, then is the same case as before. If p_j writes `X=j` after and finds it at j then it enters its CS and p_i is blocked. If p_j doesn't find `X` equal to j then it must have written `Y` before p_i and is blocked until p_i finishes the CS.

Proof of liveness:
Let p_i invock lock, where it can be blocked forever:

1. In the second wait `Y=`$\perp$: In this case there is a process p_j that wrote `Y=j` the last and it will eventually win, freeing p_i.
2. In the wait $\forall k$ `FLAG[k]=false`: This wait cannot block a process forever because every process that doesn't enter the CS puts its flag down and the one that enters the CS puts its falg down after exiting.
3. The first wait `Y=`$\perp$: In this case there is a process p_j that wrote `Y=j` before and it must eventually win because unless it must be blocked as well, but we already proved that this is not possible.

1.3.4 Deadlock freedom to Bounded bypass

Let DFL be a deadlock free protocol for mutex, we can turn it in a bounded bypass protocol using the **Round Robin** algorithm:

Algorithm: Round Robin

```
// Inizialize FLAG[i]=false, TURN as a random process id
def lock(i):
    FLAG[i]=true
    wait (TURN=i OR FLAG[TURN]=false)
    DFL.lock(i)
    return

def unlock(i):
    FLAG[i] = 0
    if FLAG[TURN]=false :
        TURN=(TURN+1)% n
    DFL.unlock(i)
    return
```

Deadlock freedom of RR

If at least one process invokes lock, then at least one process enters the CS.

Proof:

The only way to that being false is if every process is locked in the wait before invocking `DFL.lock(i)`. If `TURN=k` and p_k invokes the lock then it exit the wait, otherwise its flag is down and he other processes exit their wait.

Maximum number of iterations

If `TURN=i` and `FLAG[i]=true`, then p_i enters the CS in at most $n-1$ iterations.

Proof:
The turn changes only when `FLAG[i]` is down, if `FLAG[i]` is up p_i is in its CS or p_i is competing for its CS and will eventually invokes `DLF.lock`. If p_j invokes lock after that `FLAG[i]` is set, p_j blocks in its wait.

Let Y be the set of processes competing for the CS (suspended on `DLF.lock`). We have that $i \in Y$ and Y can't grow anymore. DLF is deadlock free so eventually some process must win and exit from Y, in the worst case all process are in Y and p_y enters the CS after all of them ($n-1$ processes).

Maximum number of turns

If `FLAG[i]` is up, then `TURN` is set to i in at most $(n-1)^2$ iterations.

Proof:
Given a random process p_i that inokes lock, when `FLAG[i]` is set the turn is of another process p_j. By the previous lemma p_j wins after at most $n-1$ iterations and the turn is increased. The worst case scenario is if the the index of p_j is equal to $i+1$, so p_i needs to wait at most $(n-1)$ turns, each with at most $(n-1)$ iterations, for a total of $(n-1)^2$ iterations. This means that in total the bound of this algorithm is $n(n-1)$.

1.4 Special HW primitives

Operating systems usualy provides specialized HW primitives, that perform in an atomic way the combination of some instrction. Using this primitives is very easy to solve the mutex problem.

1.4.1 Test&Set

Let X be a boolean register, the **Test&Set** read the register and sets it to 1. Is implemented as follows:

Algorithm: Test&Set

```
def X.Test&Set():
    tmp = X
    X = 1
    return tmp
```

Using this algorithm the mutex can be implemented using a simple protocol:

Algorithm: Mutex with Test&Set

```
// Inizialize X at 0
def lock(i):
    wait (X.Test&Set = 0)
    return

def unlock(i):
    X = 0
    return
```

1.4.2 Swap

Let X be a register, the **Swap** read the register and sets it to an input value v. Is implemented as follows:

Algorithm: Swap

```
def X.Swap(v):
    tmp = X
    X = v
    return tmp
```

Using this algorithm the mutex can be implemented using a simple protocol:

Algorithm: Mutex with Swap

```
// Inizialize X at 0
def lock(i):
    wait (X.Swap(1) = 0)
    return

def unlock(i):
    X = 0
    return
```

1.4.3 Compare&Swap

Let X be a register, the **Compare&Swap** check if the register is equal to an input value *old* and sets it to an input value *new*. Is implemented as follows:

Algorithm: Compare&Swap

```
def X.Compare&Swap(old, new):
    if X = old :
        X = new
        return true
    return false
```

Using this algorithm the mutex can be implemented using a simple protocol:

Algorithm: Mutex with Compare&Swap

```
// Inizialize X at 0
def lock(i):
    wait (X.Compare&Swap(0, 1) = True)
    return

def unlock(i):
    X = 0
    return
```

1.4.4 Fetch&Add

Let X be a register, the **Fetch&Add** read the register and to it an input value v. Is implemented as follows:

Algorithm: Fetch&Add

```
def X.Fetch&Add(v):
    tmp = X
    X = X+v
    return tmp
```

Using this algorithm the mutex can be implemented using a simple protocol:

Algorithm: Mutex with Swap

```
// Inizialize TICKET and NEXT at 0
def lock(i):
    my_tick = TICKET.Fetch&Add(1)
    wait (my_tick = NEXT)
    return

def unlock(i):
    NEXT = NEXT+1
    return
```

1.5 Safe registers

Atomic Read/Write registers and the HW primitives provide a form of atomicity. From now we will consider registers with no atomicity, the Safe registers. A safe register provides read and write in such a way:

- Every read that does not overlap with a write returns the value stored in the register
- A read that overlaps with a write return any value
- In a MRMW when two write overlap the register can contain any value

This is the weakest type of registers useful for concurrency.

1.5.1 Bakery algorithm

Asimple protocol that works with safe registers is the Bakery algorithm by Lamport:

Algorithm: Bakery algorithm

```
// Inizialize FLAG[i] = false and MY_TURN[i] to 0
def lock(i):
    // Start doorway
    FLAG[i] = true
    MY_TURN[i] = max(MY_TURN[0],...,MY_TURN[n])+1
    FLAG[i] = false
    // End doorway
    // Start bakery
    for j≠i :
        wait FLAG[j] = false
        wait (MY_TURN[j] = 0 or [MY_TURN[i],i] < [MY_TURN[j],j])
    // End bakery
    return

def unlock(i):
    MY_TURN[i] = 0
    return
```

Turn order

Let p_i be in the CS and p_j is in the doorway or the bakery, then:

$$[\texttt{MY_TURN[i]}] < [\texttt{MY_TURN[j]}]$$

This means that the mutex works as intended.
Also the algorithm has the bounded bypass property with bound $n-1$.

Proof of safety:
If p_i is in the CS it has passed the waits for j, the read of `FLAG[j]` bone by p_i in respect to the execution of p_j can be:

- The read overlaps with `FLAG[j] = t`, then p_j will read the value written in `MY_TURN[i]` and will take an higher number.
- The read overlaps with p_j being in the bakery, so the value for `MY_TURN[j]` has been set and p_i has evaluated that [`MY_TURN[i]`]¡[`MY_TURN[j]`].

Proof of liveness:
By contraddiction, assume that there is a lock but no process enters the CS. Where all the process are blocked:

- The doorway cannot block a process forever, so the first wait cannot block forever aswell.
- For the second wait the turn are totally ordered, so there is a process p_i for which the turn is the minimum that will enter the CS.

The ticket ordered means that a process cannot surpass another process 2 times, so at most a process could wait $n-1$ processes before enter the CS.

1.5.2 Aravind algorithm

In the Lamport algorithm the registers are unbounded and this could be a problem. So, another algorithm was proposed with a reset method for the tickets:

Algorithm: Aravind algorithm

```
// Inizialize FLAG[i] = false, STAGE[i] to 0 and DATE[i] to i
def lock(i):
    FLAG[i] = true
    do:
        STAGE[i] = 0
        wait (∀j ≠ i FLAG[j] = false or DATE[i] < DATE[j])
        STAGE[i] = 1
    while ∀j ≠ i STAGE[i]=0
    return

def unlock(i):
    tmp = max(DATE)+1
    if tmp ≥ 2n :
        for ∀ j :
            DATE[j] = j
    else :
        DATE[i] = tmp
    STAGE[i] = 0
    FLAG[i] = false
    return
```

Aravind algorithm properties

Two processes cannot be in the CS simultaneously, ensuring that the mutex works. Also the algorithm has the bounded bypass property with bound $2n - 2$.

Proof of safety:

Let us consider the execution of p_i leading to its CS. There is another process p_j for which p_i has done the last read of `STAGE[j]=0` before entering the CS. To enter the CS p_j has to write `STAGE[j]=1`, so its write must overlap with the read of p_i. Then p_j will read `STAGE[i]` and found it at 1, so it will not enter the CS.

Proof of liveness:

Let p_i invock the lock and let us consider the moment after the reset of `DATE` occurs. Another process p_j that enters the CS before p_i cannot surpass it again because `DATE[j]` will be higher than `DATE[i]`. In the worst case p_i could be the n-th process to enter the CS after the reset, but cannot experience two reset during its lock.

If p_n invokes the lock alone and sets `DATE[i]` to $n + 1$, then all process lock simultaneously there will be a reset done by p_{n-1} after p_n has waited $n - 1$ processes. After the reset it will wait the other $n - 1$ processes to enter the CS (because it has the higher `DATE` after the reset), making the bound $2n - 2$.

The algorithm could be revisioned changing the unlock function as follows:

Algorithm: Aravind unlock revisioned

```
def unlock(i):
    for ∀ j :
        if DATE[j]>DATE[i] :
            DATE[j] = DATE[j]-1
    DATE[i] = n
    STAGE[i] = 0
    FLAG[i] = false
    return
```

This new version lowers the bound to $n - 1$.